Thadomal Shahani Engineering College, Bandra

Name: Vansh Sharma	Subject: Agile Development & DevOps
T.E. Batch: T22 Sem: 5th Roll No: 2401109	Date: 27th August 2026

Department of Artificial Intelligence & Data Science

Experiment No. 4 (Part C)

Connecting a GitHub Repository to Jenkins for Continuous Integration

Aim

To connect a GitHub repository to Jenkins and configure a Pipeline job that automatically checks out and builds source code from the repository, and to troubleshoot a real source-control configuration error encountered during setup.

LO Mapped

LO3

Theory

Connecting Jenkins to a hosted Git repository such as GitHub allows a Pipeline job to pull the latest source code automatically instead of relying on code copied onto the Jenkins server manually. This is done by pointing the job's Pipeline definition to "Pipeline script from SCM," selecting Git as the SCM, and supplying the repository URL along with a credential Jenkins can use to authenticate (for example, a personal access token, or a GitHub App registered as a Jenkins credential). Jenkins can then be told to check for new commits either through an inbound webhook from GitHub or through periodic polling (Poll SCM) configured with a cron-style schedule. A common configuration error at this stage is a branch-name mismatch: Jenkins defaults to looking for a branch called master, while most modern GitHub repositories use main as the default branch, which causes the checkout step to fail even though the credentials and repository URL are correct.

Steps Followed

I. Jenkins Dashboard: began from a clean Jenkins dashboard at http://localhost:8080, confirming no jobs or builds were queued before creating a new Pipeline item.

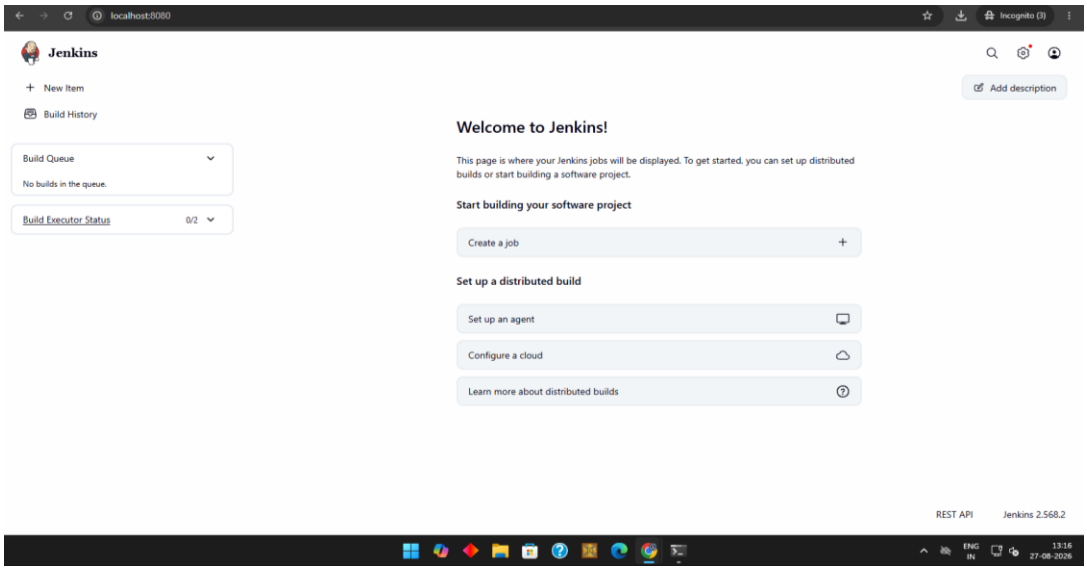


Fig. 7: Jenkins dashboard before the new pipeline job was created

II. Exploring Credential Options: while configuring source-control access for the pipeline job, the "Add GitHub App" credential type was explored in Jenkins Credentials as an alternative authentication method to a personal-access-token credential, using an App ID and private key instead of a username/token pair.

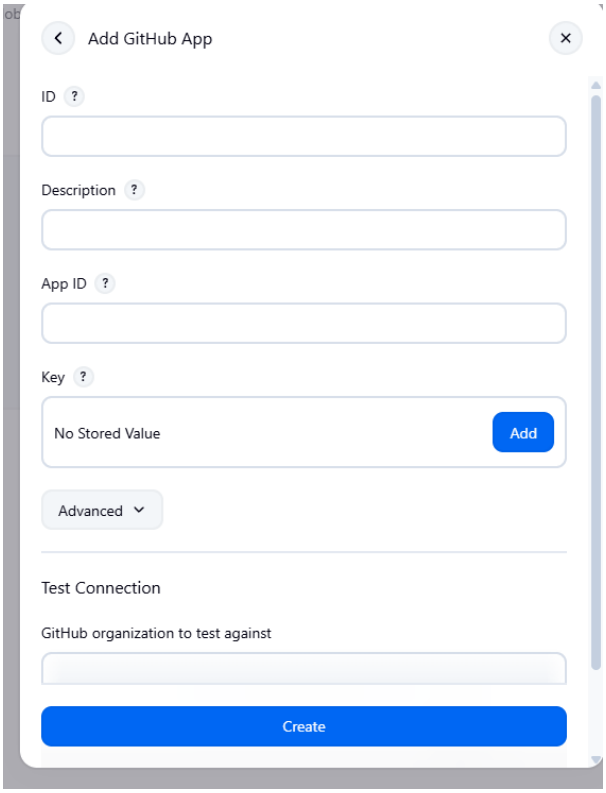


Fig. 8: "Add GitHub App" credential form in Jenkins

III. Pipeline Job Creation: a new Pipeline job named shivamkipipeline was created, with its Pipeline definition set to "Pipeline script from SCM," SCM set to Git, and the repository URL pointed at the GitHub project.

IV. First Build Failure: Build #1 was triggered and failed with a GitException. The console output showed that Jenkins attempted to run git fetch --tags --force --progress --prune -- origin +refs/heads/master:refs/remotes/origin/master, which returned the error fatal: couldn't find remote ref refs/heads/master.

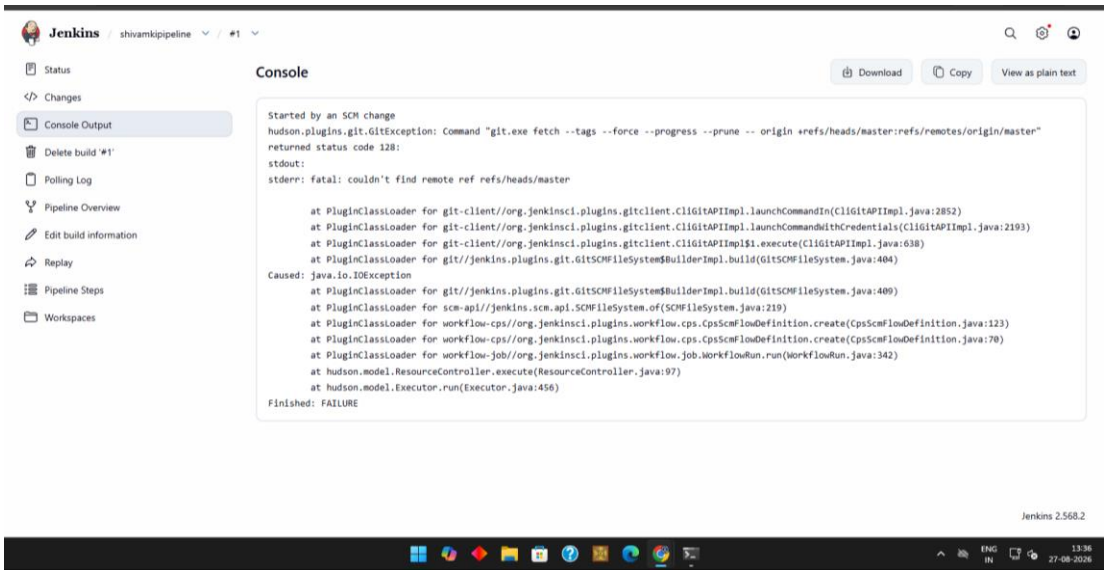


Fig. 9: Console output of Build #1 failing to find refs/heads/master

V. Dashboard Reflected the Failure: back on the Jenkins dashboard, the shivamkipipeline job listing showed a red failure indicator and a storm-cloud "weather" icon, both reflecting the recent build failure, with Last Success showing N/A.

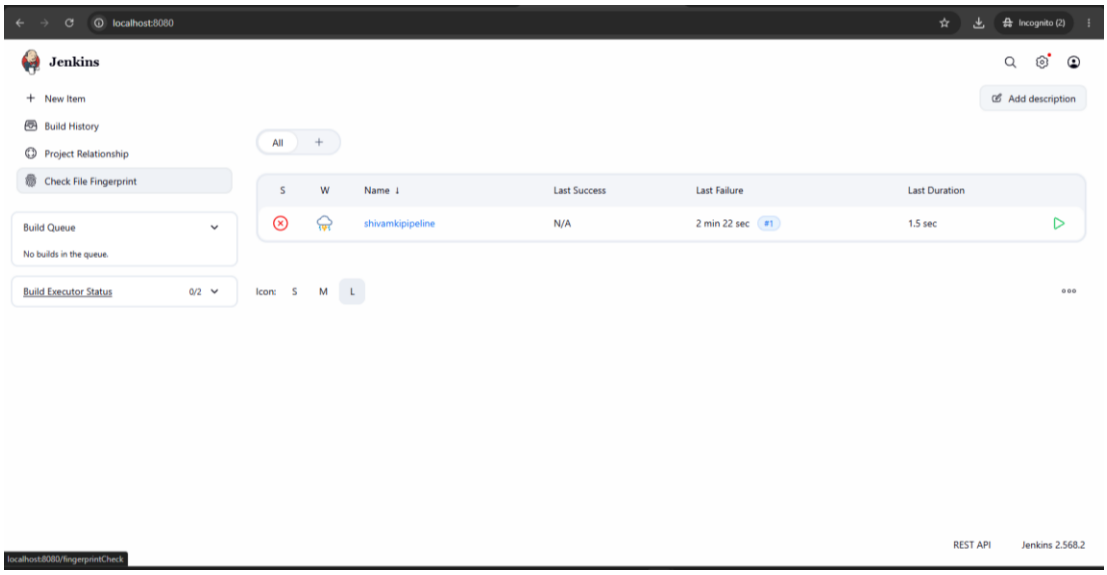


Fig. 10: Job list showing shivamkipipeline with a failed Build #1

VI. Root Cause and Fix: the GitHub repository's default branch was found to be main rather than master. The job configuration's "Branches to build" field was updated from */master to */main to match the repository.

VII. Successful Checkout on Build #2: Build #2 was triggered after the fix. The build status page confirmed a successful checkout, listing the repository as https://github.com/Yash-Singh18/odoo-

hackathon-2026, the resolved branch as refs/remotes/origin/main, and revision 74350b391 25e20410d18db0c5b237dbaa5c7693, confirming the branch-name issue was resolved.

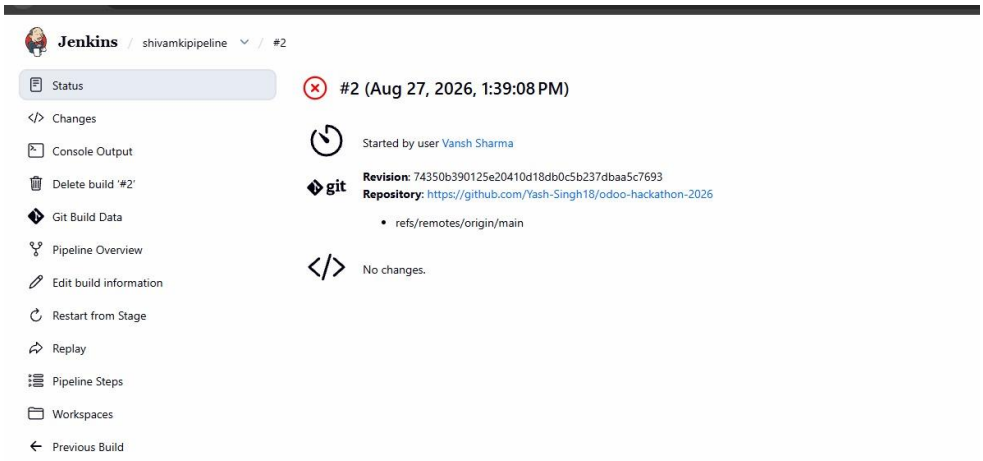


Fig. 11: Build #2 status page showing a successful checkout from refs/remotes/origin/main

Conclusion

By this assignment, a GitHub repository was successfully connected to Jenkins through a Pipeline job configured with “Pipeline script from SCM.” Along the way, an alternative GitHub App credential option was explored, and a genuine configuration error — a mismatch between Jenkins' default master branch expectation and the repository's actual main branch — was diagnosed directly from the console output and resolved by updating the job's branch specifier, after which Jenkins successfully checked out the correct revision from the repository.